

Matching and Wiring

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 2

What you need . . . 3

Conventions . . . 3

I: The Why . . . 4

1. Software is matching and wiring . . . 5

1.1. Matches the compiler makes, and checks . . . 5

1.2. Matches nobody checks . . . 6

1.3. What this module does . . . 6

2. Type, not class . . . 8

2.1. What survives erasure . . . 8

2.2. Prior art . . . 9

2.3. A trained eye . . . 9

2.4. Where the check happens . . . 10

3. The JVM, not Java . . . 11

3.1. Other platforms, with respect . . . 11

3.2. JavaScript as another bytecode . . . 12

3.3. Fall back to the keep . . . 13

3.4. Where enterprise systems are . . . 13

4. A small foundation . . . 15

4.1. Why these three . . . 15

4.2. What is in the module . . . 16

II: The How . . . 17

5. Getting started . . . 18

5.1. Adding the module . . . 18

5.2. Access . . . 18

5.3. Services . . . 18

6. Types . . . 20

6.1. Type tokens . . . 20

6.2. Assignability . . . 20

6.3. Type arguments of declarations . . . 21

6.4. Ground types . . . 22

7. Adaptation . . . **24**
7.1. Declaring adapters . . . 24
7.2. The object itself . . . 25
7.3. Methods and context . . . 25
7.4. The rules . . . 26
7.5. Plain objects . . . 27
7.6. Adaptation in code . . . 27
7.7. Generic classes . . . 28
7.8. Before there is an instance . . . 29
7.9. Choosing what to ask for . . . 29
8. Capabilities . . . **31**
8.1. A factory by hand . . . 31
8.2. Factories from methods . . . 32
8.3. Requirements are values . . . 34
8.4. Explaining a resolution . . . 34
8.5. Cycles . . . 35
8.6. Composition . . . 36
9. Flows, the pump and the transformer . . . **37**
9.1. Flows . . . 37
9.2. The pump . . . 38
9.3. Stalls . . . 38
9.4. Wiring by subscription . . . 39
9.5. Factories from methods . . . 40
10. Resource sets . . . **42**
10.1. A configured resource set . . . 42
10.2. Contributors . . . 43
10.3. Globals . . . 44
10.4. Resources from capabilities . . . 44
11. Telemetry . . . **45**
11.1. One instance per unit of work . . . 45
11.2. Watching spans live . . . 45
11.3. Resources . . . 47
11.4. Propagation . . . 48
11.5. Keys and names . . . 48
III: Looking Ahead . . . **49**
12. What is built on it, and what comes next . . . **50**

12.1. Built on it . . . 50
12.2. Planned . . . 50
Appendix A: Packages and types . . . **52**
A.1. `org.nasdanika.sdk.runtime.common` . . . 52
A.2. `org.nasdanika.sdk.runtime.common.capability` . . . 52
A.3. `org.nasdanika.sdk.runtime.common.capability.emf` . . . 53
A.4. `org.nasdanika.sdk.runtime.common.flow` . . . 53
A.5. `org.nasdanika.sdk.runtime.common.services` . . . 53
A.6. `org.nasdanika.sdk.runtime.common.telemetry` . . . 54
Appendix B: Building this book . . . **55**
Appendix C: Glossary . . . **56**

Colophon

Matching and Wiring: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the common module of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime:common:2026.11.0`, Java module `org.nasdanika.sdk.runtime.common`. Every listing in it is included from the module's sources and tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Matching and Wiring: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Java is a trademark of Oracle and its affiliates. Eclipse and EMF are trademarks of the Eclipse Foundation. OpenTelemetry is a trademark of the Linux Foundation. TypeScript and .NET are trademarks of Microsoft Corporation. Node.js is a trademark of the OpenJS Foundation. Python is a trademark of the Python Software Foundation. Kotlin is a trademark of JetBrains. Spring is a trademark of Broadcom. Other names may be trademarks of their owners. Their use here identifies the technologies discussed and implies no endorsement.

The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/runtime, in `common`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about connecting parts of a program and knowing that the connections are right. It describes `common`, the module at the bottom of the Nasdanika SDK Runtime, which every other module depends on. Its job, in one sentence, is to match things at run time the way the Java compiler matches them at compile time: by declared generic types, with the compiler's rules, and with an explanation when a match fails.

The first part is the why: why so much of software development is matching and wiring, why types rather than classes, why the JVM, and why the foundation is small. The second part is the how, a chapter per package of the module. The third says what is built on it and what comes next.

Who this book is for

Java developers who assemble applications from parts: plugins, services, handlers, factories. If you have registered a service with `ServiceLoader`, written a Spring configuration, or looked up something in a `Map<String, Object>` and cast it, the book is about the same problem. Read it in order.

Developers on other JVM languages, Kotlin, Scala and Groovy, whose compilers write the same generic declarations into class files that this module reads (Chapter 3).

Architects and technical leads choosing a platform for systems that people and AI assistants build together. Part one is for you, and needs no code.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. The runtime has a book of its own, *Nasdanika SDK Runtime: More Structure with Less Effort*, whose chapters on generic types, flows, capabilities and telemetry cover parts of this module from the runtime's side. You do not need it to read this one: where the books touch, this one repeats what you need, and goes deeper into the module.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do

with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The module is `org.nasdanika.sdk.runtime.common`, Maven coordinates `org.nasdanika.sdk.runtime:common:2026.11.0`.
- Familiarity with Java generics: the book builds on what you already know about them, which is the point of Chapter 2.

Conventions

- **Every listing is included from the module's sources or tests** and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: PLANNED for designed but not built, SKETCH for partly built.
- **Voice.** I write in the first person. Where a claim rests on my experience rather than on published research, I say so.



Editor's notes record inconsistencies between the module and its documentation found while writing, and claims to verify before publishing. They are shown in builds for the author and left out of published editions.

Part I: The Why

Why so much of software development is matching parts and checking that they fit, why matching on types rather than classes, why the JVM, and why the foundation is small. No code is needed to read this part; the few listings in it show the point to developers.

Chapter 1. Software is matching and wiring

Look at what a program is made of and most of it is connections. A method calls another method, a class implements an interface, a field holds a collaborator, a plugin answers a request, a file is opened by a parser, a URL is routed to a handler. Writing the parts is half the work. The other half is connecting them, and every connection is a **match**: something that needs, something that provides, and a rule that decides whether they fit.

Once the matches are made, they are **wiring**: the structure of references through which the program runs. A program is right only if its wiring is right, so every match needs a check, and the earlier and cheaper the check, the better.

1.1. Matches the compiler makes, and checks

The most familiar match is a method call. `library.find("Dune")` is matched to a method by its name, by the number of arguments, and by their types; when several methods fit, the compiler picks the most specific one, by rules that fill a long section of the Java Language Specification, 15.12. In the class file the call is a reference to a method by its owner, name and descriptor, and the JVM links it by matching those.

Generics are a check on that match, and a check only. After erasure, a call to `get` on a `List<Book>` is a call to `List.get`, which returns `Object`, followed by a cast the compiler inserts. The type argument is gone from the instruction. What it bought is a proof, made at compile time, that the cast will succeed: the compiler checked that only books were ever put into the list. Generics validate matches that the JVM makes without them.

Overriding is matched too, by signature, and it has a famous failure: a method meant to override another, with a typo in its name or a parameter of a slightly different type, silently becomes a new method, and the old one keeps running. `@Override` exists for that. It changes nothing in the program; it states an intent, "this method matches a method of a supertype", and the compiler checks that there is one. A match made by a rule, and validated by a declaration that a tool checks: that is the pattern this book is about.

1.2. Matches nobody checks

The compiler checks the matches it makes. Many matches are made later, at run time, by code that assembles a program from parts:

- `ServiceLoader` matches a request for an interface to the providers that modules declare.
- Dependency injection containers match injection points to beans by type, name and qualifier.
- Plugin registries match extensions to extension points, often by string identifiers.
- EMF matches a URI to a resource factory by its extension or protocol, and to a URI handler by its scheme.
- Configuration matches keys in files to properties in code by name.

These matches are made with less than the compiler has. `ServiceLoader` matches on a `Class`, so a request for a `Supplier` is answered by every supplier, whatever it supplies. Registries keyed by strings match on spelling. A `Map<String, Object>` matches on nothing at all, and the cast that follows is a hope.

When such a match is wrong, the failure is late and vague: a `ClassCastException` far from the cause, a null where a collaborator should have been, two candidates where one was expected, or a program that waits forever because two parts each wait for the other. The question "why was this wired to that?" usually has no answer short of a debugger.

1.3. What this module does

The common module makes run-time matching work like compile-time matching, in three ways.

It matches on what the compiler checked. Requests and offers are declared types: the generic return type of a method, the declared type of a field, the generic supertypes of a class. The compiler has checked every one of them, so a run-time match on them inherits the compiler's guarantee that the declaration is true.

It matches by the compiler's rules. `Supplier<Book>` satisfies `Supplier<? extends Publication>` and not `Supplier<Publication>`, at run time as in an assignment, because the same rules decide (Chapter 6).

It explains. A resolution can be traced step by step, which factory handled which requirement

and which declined, and a resolution that cannot finish fails at once with the cycle or the wait that stopped it, named (Chapter 8, Chapter 9).

The module matches in several places, all of them with the same rules:

Where	What is matched	Against what
Adaptation (Chapter 7)	A requested type, with context values	The declared types of an object's `@Adapter` fields and methods, and the object's own generic supertypes
Capabilities (Chapter 8)	A requirement: a type and payloads	The generic return types and parameter types of `@Capability` methods, or a factory's own `canHandle`
Transformation (Chapter 9)	A source object	The parameter types of `@Mapping` methods, most specific first
Resource sets (Chapter 10)	A URI	URI handlers by scheme, resource factories by extension and protocol, packages by namespace URI
Telemetry (Chapter 11)	A unit of work	The OpenTelemetry instance current in its context, which every instrumented match reports to

The last row is not a match of types, but it belongs in the table: each of the matches above runs in a span, so what was matched, when and how long it took can be watched live or read afterwards.

Chapter 2. Type, not class

Run-time wiring in Java has mostly matched on `Class`. `ServiceLoader.load(Supplier.class)` returns every supplier in the assembly. A registry keyed by class has one entry for `Supplier`, not one for a supplier of books and another for a supplier of authors. A class is the erased type: what is left of `Supplier<Book>` when the type argument is gone.

`java.lang.reflect.Type` is the rest. Its implementations describe what a Java program can declare: a class, a parameterized type such as `Supplier<Book>`, a wildcard such as `? extends Publication`, a type variable, a generic array. Matching on `Type` instead of `Class` takes run-time matching to the level the compiler works at, and the difference is the difference between "a supplier" and "a supplier of books".

2.1. What survives erasure

Objects carry no type arguments. An `ArrayList<String>` and an `ArrayList<Integer>` are the same class at run time, and nothing can recover the difference from the objects.

Declarations are different. The compiler writes the generic signature of every class, field and method into the class file, in an attribute named `Signature`, and reflection reads it back:

Listing 1. Instances forget, declarations remember (`TypesTests.java`)

```
static class Library {

    final Supplier<Book> books = () -> new Novel("Dune");

}

List<String> names = new ArrayList<>();
List<Integer> numbers = new ArrayList<>();
assertThat(names.getClass()).isSameAs(numbers.getClass()); ①

Field books = Library.class.getDeclaredField("books");
assertThat(books.getType()).isSameAs(Supplier.class); ②
assertThat(books.getGenericType())
    .isEqualTo(new TypeToken<Supplier<Book>>() {}.getType()); ③
```

① The two lists are instances of the same class.

② The field's class is the erased type.

③ The field's generic type is `Supplier<Book>`, as declared, read from the class file.

That gives the module its one rule: **what is matched is always a declaration, never an instance**. A capability method is matched by its declared generic return type, an adapter field by its declared type, an object by the generic supertypes its class declares. The compiler has checked each declaration against the code that implements it, so the run-time match inherits the compiler's guarantee.

2.2. Prior art

Matching on generic types at run time is not new, and I did not invent it.

- Neal Gafter described *super type tokens* in 2006: an anonymous subclass of a generic class records its type argument in its own generic superclass, where reflection can read it. `TypeToken` in this module is that idea, as are Guice's `TypeLiteral` and Guava's `TypeToken`.
- Guice binds and injects by `Key`, which is a `TypeLiteral` and an optional annotation, so `Supplier<Book>` and `Supplier<Author>` are different keys.
- Spring has resolved generic types with `ResolvableType` since version 4.0, and uses type arguments as implicit qualifiers when it injects.
- gentyref, and its maintained fork geantyref, implement Java's subtyping rules for reflected types. This module uses geantyref and hides it behind its own `Types` class.

What the module adds is consistency: one set of rules, used by every match in the runtime, with no string names and no annotations beyond the ones that mark what takes part.

2.3. A trained eye

There is a hypothesis in evolutionary biology that describes how this design works on the people who use it. The **sensory bias hypothesis**, also called sensory exploitation, holds that a receiver's preferences can evolve first in a context that has nothing to do with mating, such as finding food, and be exploited later by senders signaling to mates. The usual illustration is a butterfly whose eyes were tuned by natural selection to the colors of the flowers it feeds on, and a mate that later evolves wings of those colors: the preference came first, and the display took advantage of it. One of the documented cases is the guppy: females are attracted to orange objects, a preference that helps them find fruit, and males carry orange spots.

Java developers have a trained eye for generic types. It was trained by collections, from Java 5 on:

twenty years of `List<Book>` and `Map<String, Author>` have taught every Java developer to read a type argument, to know why a `List<Novel>` is not a `List<Book>`, and to reach for `? extends` when only reading. That skill evolved for type-safe collections, not for wiring. Nasdanika piggybacks on it. A requirement for `Supplier<? extends Publication>` asks for exactly what a Java developer reads in it, and is answered by exactly what the compiler would accept, so there is no new notation to learn: no qualifiers, no string identifiers, no wiring files.

Like any analogy, this one has limits. Evolution has no intent, and the runtime's design does; and nobody is being deceived, since the display delivers what it promises. What carries over is the order of events: the preference already exists, and the design takes advantage of it rather than asking for a new one.



> Verify the biology before publishing. The sensory bias hypothesis is usually credited to Basolo (1990, platyfish and swordtails) and Ryan (1990); the guppy case is Rodd, Hughes, Grether and Baril, "A possible non-sexual origin of mate preference: are male guppies mimicking fruit?", *Proceedings of the Royal Society B*, 2002. The butterfly is used here as an illustration, not as a documented case; find a study of butterflies, or keep the wording that says it is an illustration.

2.4. Where the check happens

The trained eye is half of it; the other half is the compiler. A declaration such as `Supplier<Book> books()` is checked twice, at two different times. At compile time the compiler checks that the method returns what it says, which is what lets the runtime trust the declaration. At run time the module checks that the declaration fits what is asked for, with the same rules the compiler uses for an assignment. Neither check is new; what is new is that the second one uses the first.

Chapter 3. The JVM, not Java

What the module matches on is in the class file, not in Java source. The `Signature` attribute is part of the JVM specification, and every compiler that targets the JVM can write it. Kotlin, Scala and Groovy do, for the types their programs declare, so a Kotlin class that implements `Supplier<Book>` is matched exactly like a Java one, and a capability method written in Groovy with a declared generic return type answers the same requirements as its Java twin.

There are differences at the edges. Kotlin writes its declaration-site variance as wildcards in some positions and not in others, and Groovy declares `Object` wherever a dynamic program leaves a type out, so such a member matches only requirements for `Object`. The rule is the same in every language: what is matched is what was declared, and a member that declares nothing answers nothing in particular.

So this is a book about the JVM. Java is the language its examples are written in.

3.1. Other platforms, with respect

The two platforms most often weighed against the JVM for new systems are Node.js, programmed in TypeScript or JavaScript, and Python. Both are excellent at what they are designed for, and both have type systems worth admiring. Neither has what this module is built on: declarations that a compiler has checked and that the running program can read.

TypeScript has a type system richer than Java's in many ways: structural types, unions, conditional and mapped types. It is checked at compile time and then erased completely; the JavaScript that runs carries no types at all. With the `emitDecoratorMetadata` option the compiler records some types for decorated members, and dependency injection frameworks such as Angular and NestJS use them, but what it records is constructors: a `Book[]` is recorded as `Array`, and an interface as `Object`. Matching on a declared `Supplier<Book>` at run time is not possible without a second declaration, a schema in a library such as Zod, that a developer keeps in step with the first.

Python keeps annotations at run time, generic arguments included: `typing.get_type_hints` returns `list[Book]` for a parameter declared that way, and FastAPI and Pydantic build very useful matching on it. What Python does not do is check them. The interpreter neither verifies that a function returns what its annotation says nor complains when it does not; checkers such as

mypy and Pyright do, when somebody runs them, and they are not part of the program. Matching on an annotation nobody has checked is matching on a claim.

.NET, to be fair, goes further than the JVM: its generics are reified, so even instances know their type arguments. This book is about the JVM because the runtime is, not because the JVM is the only platform with checked generic declarations.

3.2. JavaScript as another bytecode

The browser runs JavaScript, and that will not change. It does not follow that JavaScript has to be what people write. Scott Hanselman called JavaScript the assembly language of the web in 2011, and compilers from other languages to JavaScript are older than that: Google Web Toolkit compiled Java to JavaScript in 2006, and J2CL, TeaVM, Kotlin/JS and Scala.js do it today.

The runtime takes a different route to the same place. It has models of TypeScript, React and HTML, and generators that write readable modules and pages from them, with imports computed from references. A web application is built in Java, from models, and what reaches the browser is JavaScript that the generator wrote. In that arrangement JavaScript is a generation target, another bytecode: the format a machine executes, written by a tool from a source people work in.

That raises a question of direction, and it deserves a careful answer. On the JVM, with these generators, a team can do most of what it would do with Node.js to build and serve a browser application: generate the modules and pages, serve them, test them, deploy them as static files. Not all of it. Bundlers, server-side rendering of React, and the npm tool chain of linters and component test runners run on Node.js; some of them run on the JVM through GraalJS, and the rest stay where they are. The other direction is possible as text: a Node.js program can write Java source as easily as Java writes JavaScript. What it cannot do is be the JVM: it has no checked generic declarations to match at run time, and it cannot load the JVM’s libraries, EMF, JDBC drivers and the rest of the enterprise ecosystem, into its own process. So the asymmetry is real, but it is narrower than "the JVM can do everything Node.js can": the JVM can generate and host most of a browser application, while Node.js can generate Java but cannot host what Java runs on.

3.3. Fall back to the keep

In *The Two Towers*, when the outer wall of Helm's Deep is breached, the defenders fall back to the keep of the Hornburg, the innermost and strongest part of the fortress; in the film the order is shouted along the wall: "Fall back to the keep!"

The analogy is not literal. AI assistants are not the enemy; they are allies, and very capable ones. But the shape of the situation is similar. Assistants now hold most of the outer wall: they write code, tests and documentation faster than people can read them. What people keep is the place from which they can still check what was built, and the stronger that place, the more of the work can be given to the assistants safely.

I think the JVM is a better keep than Node.js for that purpose, and the reasons are the ones in this book. Its declarations are checked by a compiler and kept in the class files, so a person or a program can read what a part claims to provide and rely on the claim. Its run-time matches can use the same declarations and the same rules, and explain themselves. And with JavaScript as a generation target, retreating to the keep gives up nothing in the browser.

3.4. Where enterprise systems are

My own experience, which is not a study: in the enterprises I have worked in and with, Java has been at least twice as common as Node.js for systems in production, and about ten times as common as Python.

I know of no public research that measures this across enterprises, and the surveys that are published measure something else. Developer surveys count developers and the languages they use, not systems and what they run on, and by that measure JavaScript and Python are used by more professional developers than Java. Both pictures can be true at once: many developers write some JavaScript and some Python, while, in my experience, the systems of record of large organizations, the ones that run payments, policies and supply chains, run disproportionately on the JVM.



> Verify or remove before publishing. The ratios are the author's experience. Survey data found while writing: the Stack Overflow Developer Survey 2025 (survey.stackoverflow.co/2025/technology/), as summarized in secondary write-ups, has about 69 percent of professional developers using JavaScript, about 55

> percent Python and about 30 percent Java, and Node.js used by about 49 percent of all respondents; check the figures on the survey's own page. JetBrains' State of Java 2025 (lp.jetbrains.com/the-state-of-java-2025/) was reported to show Java ahead of Python for microservices, 51 percent to 26 percent; not verified. The frequently repeated "90 percent of the Fortune 500 use Java" has no traceable primary source and should not be cited. Look for analyst research (Gartner, Forrester, IDC) on languages of production systems in large enterprises.
>
> Also verify the film line ("Fall back to the keep!" in Peter Jackson's *The Two Towers*, 2002) and Hanselman's 2011 post ("JavaScript is Assembly Language for the Web").

Chapter 4. A small foundation

Every module of the runtime depends on `common`, so whatever `common` depends on, everything does. That makes its dependencies a decision for the whole runtime, and the decision is to keep them to three:

- **The JDK**, currently Java 25.
- **EMF core**: `org.eclipse.emf.common`, `org.eclipse.emf.ecore` and `org.eclipse.emf.ecore.xmi`.
- **The OpenTelemetry API**: `io.opentelemetry.api` and `io.opentelemetry.context`, not the SDK.

All three are small, stable and standard in their space. A platform team can approve "Java, EMF and the OpenTelemetry API" in a meeting, while a longer list is a review.

The one exception is geantyref, a small library with no dependencies of its own, which implements Java's subtyping rules for reflected types. The module requires it without `transitive`, and no public signature mentions it: `Types` wraps it, so it could be replaced without changing any code that uses the module.

4.1. Why these three

The JDK first. Where the JDK has a feature, the module uses it: `java.util.concurrent.Flow` for streams of values, virtual threads for concurrency, `ServiceLoader` for discovery, `MethodHandles` for reflective calls, and the module system for encapsulation. A library earns its place by doing something the JDK does not.

EMF, because the runtime is about models. Its resource sets are how models are loaded, saved and cross-referenced, and they are matching machines too: a URI is matched to a handler, a resource to a factory, a namespace to a package. The module configures resource sets from whatever is on the module path (Chapter 10).

The OpenTelemetry API, because matches should be observable. A resolution that wires a hundred capabilities, a transformation of a model with thousands of elements, a resource loaded over the network: each is an operation someone will want to see while it runs, or read about afterwards. The API costs nothing when no SDK is installed, and the module reports through it everywhere (Chapter 11).

4.2. What is in the module

Package	What it provides
`org.nasdanika.sdk.runtime.common`	Generic type matching (`Types`, `TypeToken`), adaptation (`Adaptable`, `@Adapter`, `Adapters`), and a configured EMF resource set (`NasdanikaResourceSet`)
`⋯common.capability`	The capability framework: requirements, factories, loaders, and reflective factories assembled from `@Capability` methods
`⋯common.capability.emf`	Resources whose contents are loaded and saved by handlers found as capabilities
`⋯common.flow`	Operators for `java.util.concurrent.Flow`, the pump that runs them deterministically, and the transformer that wires object graphs
`⋯common.services`	Service interfaces for configuring resource sets, and the URI handlers and resource factories the module contributes
`⋯common.telemetry`	One OpenTelemetry instance per unit of work, instrumentation helpers, and an OpenTelemetry wrapper whose spans can be watched live

Appendix A lists the main types of each package.

Part II: The How

The module in Java: adding it to a project, matching generic types, adapting objects, resolving capabilities, wiring objects with flows, configuring resource sets, and observing all of it with OpenTelemetry.

Chapter 5. Getting started

Using the module takes a Maven dependency and a line in `module-info.java`.

5.1. Adding the module

The runtime has a bill of materials, `org.nasdanika.sdk.runtime:bom`, which manages the versions of its modules and of their dependencies. Import it in `dependencyManagement` with the scope `import` and the type `pom`, and depend on `org.nasdanika.sdk.runtime:common` without a version.

The module is a named Java module, `org.nasdanika.sdk.runtime.common`. Require it in your `module-info.java`. It requires EMF core and the OpenTelemetry API transitively, so your module reads them too, and it exports all of its packages.

5.2. Access

The module finds `@Adapter` and `@Capability` members reflectively, so the module system has to let it. Public members of exported packages are accessible to everyone. For the rest there are two ways:

- **Open the package** to the runtime, which lets it read private members such as adapter fields: `opens org.example.library to org.nasdanika.sdk.runtime.common`. A qualified `opens … to` keeps the package closed to everybody else.
- **Pass a lookup.** Reflective capability factories and transformer factories take a `MethodHandles.Lookup` created in the module that declares the methods, which grants the runtime that module's access, so the package needs to be neither exported nor open (Chapter 8).

5.3. Services

The module discovers its extensions with `ServiceLoader`, and declares what it uses and provides:

Listing 2. What the module uses and provides (`module-info.java`)

```
uses ResourceSetContributor;
uses URIHandler;
uses AdapterFactory;
```

```
uses EPackage;
uses CapabilityFactory;

provides URIHandler with
    ClassPathURIHandlerFactory,
    ModuleURIHandlerFactory;

provides ResourceSetContributor with
    BinaryResourceFactoryContributor,
    GzipBinaryResourceFactoryContributor,
    XMIResourceFactoryContributor;
```

A module extends the runtime by providing the same services: a `CapabilityFactory` for capabilities (Chapter 8), a `ResourceSetContributor` to configure resource sets, a `URIHandler` for a URI scheme, an `EPackage` for a model (Chapter 10). Declaring a provider is all the registration there is. The module's own providers are in the listing: two URI handlers, for the `classpath` and `module` schemes, and resource factories for XMI and for EMF's binary format, plain and compressed.

Chapter 6. Types

`Types` is the module's small library of generic type operations, and every match in the runtime goes through it. This chapter shows its operations on three types:

Listing 3. The types of the examples (`TypesTests.java`)

```
interface Publication {}

interface Book extends Publication {}

record Novel(String title) implements Book {}
```

6.1. Type tokens

A class literal cannot name a parameterized type: there is no `Supplier<Book>.class`. A type token can. `new TypeToken<Supplier<Book>>() {}` creates an anonymous subclass of `TypeToken`, whose generic superclass, `TypeToken<Supplier<Book>>`, is a declaration, and so survives erasure; `getType()` reads its type argument back.

Types built by the JDK, by reflection or from tokens, are equal when they describe the same type, so a type from a token can be compared with the generic type of a field, as in Chapter 2, and either can be a map key.

6.2. Assignability

`Types.isAssignable(target, source)` answers the question the compiler answers for an assignment: may a value declared as `source` be used where `target` is required? The compiler's answer first:

Listing 4. What the compiler accepts

```
Supplier<Book> books = () -> new Novel("Dune");
Supplier<? extends Publication> some = books; // Compiles
// Supplier<Publication> exact = books;        // Does not compile: incompatible types
```

and the same questions asked at run time:

Listing 5. Assignability by the compiler's rules

```
Type bookSupplier = new TypeToken<Supplier<Book>>() {}.getType();
```

```
Type publicationSupplier = new TypeToken<Supplier<Publication>>() {}.getType();
Type someSupplier = new TypeToken<Supplier<? extends Publication>>() {}.getType();
Type anySupplier = new TypeToken<Supplier<?>>() {}.getType();

assertThat(Types.isAssignable(someSupplier, bookSupplier)).isTrue(); ①
assertThat(Types.isAssignable(publicationSupplier, bookSupplier)).isFalse(); ②
assertThat(Types.isAssignable(Supplier.class, bookSupplier)).isTrue(); ③
assertThat(Types.isAssignable(bookSupplier, Supplier.class)).isFalse(); ④
assertThat(Types.isAssignable(anySupplier, Supplier.class)).isTrue(); ⑤
```

① A supplier of books is a supplier of *some* publication: the wildcard admits subtypes.

② It is not a supplier of publications. Generics are invariant: a `Supplier<Publication>` could be expected to supply an almanac.

③ Anything parameterized can be used where the raw type is required.

④ The other way round is where the module is deliberately stricter than Java.

⑤ The exception: a target whose type arguments are all unbounded wildcards.

The first three are Java's rules. The fourth is not. Java lets a raw `Supplier` be used where `Supplier<Book>` is required, with an unchecked warning. For matching, that would let something declared as a raw supplier answer a requirement for books, about which it has said nothing, so `Types` refuses. `Supplier<?>`, which Java accepts from a raw type without a warning, is the exception.

The lesson for requirements is the one the JDK's own signatures teach, as in `Collection.addAll(Collection<? extends E>)`: the side that provides declares the exact type it has, and the side that asks uses `? extends` wherever it only reads. Each wildcard admits more providers; each exact type argument insists on that type, at that level.

6.3. Type arguments of declarations

A class fixes the type arguments of its generic supertypes, and `getTypeArgument` finds them, however many classes lie in between:

Listing 6. A class that fixes a type argument two levels up

```
abstract static class Shelf<T extends Publication> implements Supplier<List<T>> {}

static class NovelShelf extends Shelf<Novel> {

    @Override
    public List<Novel> get() {
```

```
        return List.of(new Novel("Dune"), new Novel("Solaris"));
    }

}
```

Listing 7. Reading type arguments

```
Type supplied = Types.getTypeArgument(NovelShelf.class, Supplier.class.getTypeParameters()[0]);
assertThat(supplied).isEqualTo(new TypeToken<List<Novel>>() {}.getType()); ①

Type wildcard = new TypeToken<List<? extends Book>>() {}.getType();
assertThat(Types.getElementType(wildcard, List.class.getTypeParameters()[0]))
    .isEqualTo(Book.class); ②

assertThat(Types.getTypeArgument(Shelf.class, Supplier.class.getTypeParameters()[0]))
    .isNull(); ③
```

① `NovelShelf` supplies a `List<Novel>`: `Shelf<Novel>` fixes `T`, and `Shelf<T>` fixes `Supplier`'s argument as `List<T>`.

② `getElementType` is the same for containers, with a wildcard replaced by its bound: an element of `List<? extends Book>` is a `Book`.

③ `Shelf` itself is generic in `T`, so it fixes nothing, and the answer is null rather than a type variable.

Three more operations see members through a subtype: `getFieldType`, `getReturnType` and `getParameterTypes` resolve the type variables of a member's declaring class as a subclass fixes them, so a field declared as `T content` in `Box<T>` is a `Novel` in `NovelBox extends Box<Novel>`. Adaptation relies on them (Chapter 7).

6.4. Ground types

A type variable means something only inside the scope that declares it, so `T`, or `List<T>`, cannot be required or provided. `isGround` tells the two apart. This is the trap of type tokens created inside generic methods:

Listing 8. A token that captures a variable

```
static <T> Type listOf() {
    return new TypeToken<List<T>>() {}.getType(); // Captures T, not what T stands for
}

assertThat(Types.isGround(new TypeToken<List<? extends Book>>() {}.getType())).isTrue();
```

```
assertThat(Types.isGround(TypesTests.<Book>listOf())).isFalse();
```

`new TypeToken<List<T>>() {}` captures `List<T>`, whatever `T` stands for at the call site, because erasure leaves nothing else to capture. Requirements refuse such types with an error, rather than match them wrongly (Chapter 8).

Chapter 7. Adaptation

Matching types answers "does this declaration fit that one". Adaptation answers a question objects are asked all the time in a program assembled from parts: "can you give me what I need?" The asker has an object and a type; the object answers with something of that type, or with null.

The idea is old. Eclipse's `IAdaptable` and EMF's adapter factories ask it with a class. This module asks it with a type, and lets objects answer through declarations instead of code.

7.1. Declaring adapters

An object declares what it can be adapted to with `@Adapter` on fields and methods. The declared generic type of the member is what it provides:

Listing 9. Adapter fields (`AdaptationTests.java`)

```
static class Branch implements Adaptable {

    @Adapter
    private final Supplier<Book> books = () -> new Novel("Dune"); ①

    @Adapter
    private final Supplier<Publication> publications = () -> new Novel("Almanac"); ②

}
```

① The branch can be adapted to `Supplier<Book>`.

② And to `Supplier<Publication>`: another field of the same class, `Supplier`, told apart by its type argument.

Asking is `adaptTo(type, context…)`:

Listing 10. Adapting

```
Branch branch = new Branch();
Supplier<Book> books = branch.adaptTo(BOOKS);
assertThat(books.get()).isEqualTo(new Novel("Dune"));

Supplier<Publication> publications = branch.adaptTo(PUBLICATIONS);
assertThat(publications.get()).isEqualTo(new Novel("Almanac"));

Type strings = new TypeToken<Supplier<String>>() {}.getType();
```

```
assertThat(branch.<Object>adaptTo(strings)).isNull(); ①
```

① Nothing declared fits a `Supplier<String>`, so the answer is null.

`Adaptable` is an interface with one default method, so implementing it costs nothing.

7.2. The object itself

An object adapts to its own declared types, generic supertypes included, without any annotation:

Listing 11. Adapting to a generic supertype

```
static class Bookcase implements Supplier<Book>, Adaptable {

    @Override
    public Book get() {
        return new Novel("Solaris");
    }

}

Bookcase bookcase = new Bookcase();
assertThat(bookcase.<Object>adaptTo(BOOKS)).isSameAs(bookcase);
assertThat(bookcase.<Object>adaptTo(PUBLICATIONS)).isNull(); // Invariance
```

7.3. Methods and context

A method provides by its generic return type. Parameters take context: values passed after the type, matched to the parameters in order.

Listing 12. Adapter methods (`AdaptationTests.java`)

```
static class Manuscript implements Adaptable {

    private final int chapters;

    Manuscript(int chapters) {
        this.chapters = chapters;
    }

    @Adapter
    public Chapter chapter(int number) { ①
        return number <= chapters ? new Chapter(number) : null; ②
    }

    @Adapter
```

```
    public List<Chapter> chapters() { ③
        return IntStream.rangeClosed(1, chapters).mapToObj(Chapter::new).toList();
    }

}
```

① One parameter: answers a request for a `Chapter` with one value of context, an `int`.

② Returning null declines: "not this time".

③ No parameters: answers a request without context.

Listing 13. Adapting with context

```
Manuscript manuscript = new Manuscript(3);
assertThat(manuscript.<Chapter>adaptTo(Chapter.class, 2)).isEqualTo(new Chapter(2));
assertThat(manuscript.<Chapter>adaptTo(Chapter.class, 7)).isNull(); // Declined
assertThat(manuscript.<Chapter>adaptTo(Chapter.class, "2")).isNull(); // Not an int
assertThat(manuscript.<Chapter>adaptTo(Chapter.class)).isNull(); // No context

List<Chapter> chapters = manuscript.adaptTo(new TypeToken<List<Chapter>>() {}.getType());
assertThat(chapters).hasSize(3);
```

A method matches when it has as many parameters as there are context values, and each value is an instance of its parameter's class, boxed for primitives, or null for a parameter that is not primitive.

7.4. The rules

In order:

- With no context, the object itself is returned if its own declared type matches.
- Otherwise, adapter fields and adapter methods without parameters are tried; with context, adapter methods whose parameters accept the context.
- Of the candidates whose declared type is assignable to the requested one, the more specific are tried first, then fields before methods, then by name. The first non-null value wins.

Members are found once per class and remembered, so the cost of reflection is paid once.



"More specific first" is computed by `Types.specificity`, which counts the supertypes of the **erased** type. `Supplier<Book>` and `Supplier<Publication>` both erase to `Supplier`, so they tie, and the member name decides. A request for

> `Supplier<? extends Publication>` gets `books` because "books" sorts before "publications", not because it is more specific; the test `AdaptationTests.specificity` passes for that reason. The same ordering is used by `ReflectiveCapabilityFactory` and by command wiring. A fix: break ties between members whose erased types are equal by comparing their declared types with `Types.isAssignable`, so that a member assignable to the other comes first.

7.5. Plain objects

An object that does not implement `Adaptable` can be adapted through its annotations with `Adapters.adapt`:

Listing 14. A plain object

```
static class Catalog { // Not Adaptable

    @Adapter
    public final Supplier<Book> books = () -> new Novel("Neuromancer");

}

Supplier<Book> books = Adapters.adapt(new Catalog(), BOOKS);
assertThat(books.get()).isEqualTo(new Novel("Neuromancer"));
```

`Adapters.adapt` asks an `Adaptable` object itself, so its overrides apply, and reads the annotations of any other object.

7.6. Adaptation in code

Implementing `Adaptable` is needed only to adapt in ways an annotation cannot express: computed, delegated or cached. An override can fall back to the declarations:

Listing 15. An override with a fallback

```
static class Archive implements Adaptable {

    @Adapter
    private final Supplier<Book> books = () -> new Novel("Ubik");

    @Override
    public <T> T adaptTo(Type type, Object... context) {
        if (type == String.class && context.length == 1 && context[0] instanceof String key) {
```

```
            @SuppressWarnings("unchecked")
            T label = (T) ("archive:" + key);
            return label;
        }
        return Adaptable.super.adaptTo(type, context); ①
    }

}

Archive archive = new Archive();
assertThat(archive.<String>adaptTo(String.class, "ubik")).isEqualTo("archive:ubik");
Supplier<Book> books = archive.adaptTo(BOOKS);
assertThat(books.get()).isEqualTo(new Novel("Ubik"));
```

① Everything the override does not handle goes to the declared adapters.

7.7. Generic classes

A member declared with a type variable provides what the variable stands for in the object's class:

Listing 16. A type variable resolved by a subclass

```
static class Box<T> implements Adaptable {

    @Adapter
    protected final T content;

    Box(T content) {
        this.content = content;
    }

}

static class NovelBox extends Box<Novel> {

    NovelBox(Novel novel) {
        super(novel);
    }

}

assertThat(new NovelBox(new Novel("Dune")).<Book>adaptTo(Book.class)).isNotNull(); ①
assertThat(new Box<>(new Novel("Dune")).<Book>adaptTo(Book.class)).isNull(); ②
```

① In `NovelBox`, `T` is `Novel`, so `content` provides a `Book`.

② A `Box` created with a diamond carries no type arguments, because instances do not; for it, `T` is only its bound, `Object`, and an `Object` is not a `Book`.

The second line is the rule of Chapter 2 at work: declarations, never instances. To adapt by a type argument, fix it in a class.

Overrides of adapter methods stay adapters, without repeating the annotation, and the override is what is called.

7.8. Before there is an instance

Sometimes the question comes before the object exists. A command line builds its tree of commands before any command runs, and needs to know whether a command could work under a parent without creating either. `Adapters.isAdaptable(class, type, contextTypes…)` answers from declarations only:

Listing 17. Asking a class

```
assertThat(Adapters.isAdaptable(Branch.class, BOOKS)).isTrue();
assertThat(Adapters.isAdaptable(Bookcase.class, BOOKS)).isTrue();
assertThat(Adapters.isAdaptable(NovelBox.class, Book.class)).isTrue();
assertThat(Adapters.isAdaptable(Manuscript.class, Chapter.class, int.class)).isTrue();
assertThat(Adapters.isAdaptable(Manuscript.class, Chapter.class)).isFalse(); ①
assertThat(Adapters.isAdaptable(Archive.class, String.class, String.class)).isFalse(); ②
```

① `chapter(int)` needs context; without it, nothing declared answers a `Chapter`.

② An override is code, not a declaration, so the check cannot see it. An `@Adapter` method that delegates to the override makes it visible.

True means an instance can be asked and has a declared way to answer, not that it will: an adapter can still decline.

7.9. Choosing what to ask for

The requested type decides who can answer. Exact type arguments insist; wildcards admit:

Listing 18. A method that provides a list of novels

```
static class NovelList {

    @Adapter
    public List<Novel> novels() {
        return List.of(new Novel("Dune"));
    }
```

```
}

Type exact = new TypeToken<List<Book>>() {}.getType();
Type readable = new TypeToken<Iterable<? extends Book>>() {}.getType();
assertThat(Adapters.isAdaptable(NovelList.class, exact)).isFalse(); ①
assertThat(Adapters.isAdaptable(NovelList.class, readable)).isTrue(); ②
```

① A `List<Novel>` is not a `List<Book>`: someone holding a `List<Book>` could add any book to it, not only a novel.

② Anything that can be iterated for books is answered by it.

Ask for the least you need: if you only read, say `? extends`.

Chapter 8. Capabilities

Adaptation asks an object you already have. Capabilities answer the question that comes before it: where do the objects come from?

A **requirement** says what is needed. A **capability factory** says which requirements it can handle and creates **capability providers** for them, which publish the capabilities. A **capability loader** takes a requirement, asks every factory, and collects what they provide. Factories can have requirements of their own, which the loader resolves within the same resolution.

With a type and no payload, a requirement is a lookup by type: what `ServiceLoader` does, but generic, lazy, asynchronous, and able to resolve dependencies.

8.1. A factory by hand

The interface has two methods:

Listing 19. A capability factory (`CapabilityTests.java`)

```
static final CapabilityFactory<Object, Credentials> CREDENTIALS = new CapabilityFactory<>() {

    @Override
    public boolean canHandle(Object requirement) { ①
        return requirement == Credentials.class;
    }

    @Override
    public Flow.Publisher<CapabilityProvider<Credentials>> create(
            Object requirement,
            Loader loader,
            Span span,
            Logger logger,
            Meter meter) {
        return Flows.of(CapabilityProvider.of(new Credentials("vault"))); ②
    }

    @Override
    public String toString() {
        return "credentials";
    }

};
```

① A requirement can be any object. This factory handles one: the class `Credentials`.

② Providers are published, so a factory can provide many capabilities, later, or none.

The `Span` is the factory's own span in the resolution, and the `Logger` and the `Meter` report to the same OpenTelemetry instance (Chapter 11). The meter is for measuring what providing capabilities costs, such as embeddings generated or tokens spent.

Listing 20. Loading

```
CapabilityLoader loader = CapabilityLoader.of(CREDENTIALS);
List<Credentials> all = loader.loadAll(Credentials.class); ①
assertThat(all).containsExactly(new Credentials("vault"));
Credentials one = loader.loadOne(Credentials.class); ②
assertThat(one).isEqualTo(new Credentials("vault"));
assertThat(loader.<Object>loadOne(String.class)).isNull(); ③
```

① Every capability for the requirement, from every factory that handles it.

② The first, or null.

③ Nobody handles `String.class`: no capability, and no error.

In an application, factories are services: a module declares `provides CapabilityFactory with …` in its `module-info.java`, and `CapabilityLoader.getInstance()` is the shared loader over every factory on the module path.

8.2. Factories from methods

Most factories are not written by hand. A `ReflectiveCapabilityFactory` is assembled from methods annotated with `@Capability`, so one class, and one `provides` entry, contributes many capabilities:

Listing 21. A reflective factory

```
static class LibraryCapabilities extends ReflectiveCapabilityFactory {

    LibraryCapabilities() {
        super(MethodHandles.lookup()); ①
    }

    @Capability
    public Supplier<Book> books() { ②
        return () -> new Novel("Dune");
    }

    @Capability
    public Supplier<Publication> publications() {
        return () -> new Novel("Almanac");
```

```
    }

    @Capability
    public CompletionStage<Catalog> catalog(String branch, Loader loader) { ③
        return loader
            .<Credentials>loadAll(Credentials.class) ④
            .thenApply(credentials -> new Catalog(branch, credentials.get(0).vault()));
    }

}
```

① A lookup from the declaring module, so that the methods need not be in an exported package.

② What a method provides is its generic return type, checked by the compiler and matched by `Types.isAssignable`.

③ Parameters of the requirement's payload, here a `String`, in order; and injected parameters, here the `Loader`, by type.

④ A dependency, resolved within the same resolution. `CompletionStage` delivers the capability when it is ready.

Listing 22. Requirements by type and by payload

```
CapabilityLoader loader = CapabilityLoader.of(new LibraryCapabilities(), CREDENTIALS);

Type books = new TypeToken<Supplier<Book>>() {}.getType();
assertThat(loader.<Supplier<Book>>loadAll(books)).hasSize(1); ①

Requirement some = Requirement.of(new TypeToken<Supplier<? extends Publication>>() {});
assertThat(loader.<Supplier<?>>loadAll(some)).hasSize(2); ②

Requirement catalog = Requirement.of(Catalog.class, "Downtown"); ③
assertThat(loader.<Catalog>loadOne(catalog)).isEqualTo(new Catalog("Downtown", "vault"));
```

① A requirement can be a bare type. Only `books()` provides a `Supplier<Book>`.

② A wildcard admits both suppliers.

③ A requirement with a payload: a `Catalog`, for the branch "Downtown". `catalog` takes a `String`, so it matches; the other methods take no payload, so they do not.

What a method takes:

- **Injected parameters**, by type, in any position: `Loader` for dependencies, `Span` (the method's own span), `Logger`, `Meter`, `OpenTelemetry`, and `Type`, the required type, for a method that serves a range of them.

- **Payload parameters**, every other parameter, matched to the requirement's payloads in order: as many parameters as payloads, each payload an instance of its parameter's class, or null for one that is not primitive.

What a method returns:

- **The capability**, matched by its declared type.
- **A way of delivering it**: `CompletionStage<X>`, `Flow.Publisher<X>`, `CapabilityProvider<X>` or `Flow.Publisher<CapabilityProvider<X>>`. These are always unwrapped, and `X` is matched.
- **A container**: `X[]` or an `Iterable<X>`. The declared type is matched first, so a requirement for `List<X>` gets the list; only if it does not match are the elements, so a requirement for `X` gets them one by one.

Null provides nothing, which is how a method declines. A generic method, `<T> Supplier<T> make(Class<T> type)`, and a `void` method are refused when the factory is created, because their declarations say nothing that can be matched.

8.3. Requirements are values

A resolution remembers what it has resolved, keyed by requirement, so that a dependency shared by many factories is created once. That needs requirements with value equality, and `Requirement` is a record:

Listing 23. The same requirement however it was built

```
Type reflected = CapabilityTests.class.getDeclaredField("declared").getGenericType();
Requirement fromField = Requirement.of(reflected, "Downtown");
Requirement fromToken = Requirement.of(new TypeToken<Supplier<Book>>() {}, "Downtown");
assertThat(fromField).isEqualTo(fromToken).hasSameHashCodeAs(fromToken);
```

Payloads should have value equality too; records do. A requirement refuses a type that is not ground, such as `List<T>` (Chapter 6).

8.4. Explaining a resolution

A loader with a listener reports what happens during a resolution, and `ResolutionListener.explaining` turns the events into lines of text:

Listing 24. Asking for an explanation

```
List<String> lines = new ArrayList<>();
CapabilityLoader loader = CapabilityLoader
    .of(new LibraryCapabilities(), CREDENTIALS)
    .withListener(ResolutionListener.explaining(lines::add));
loader.loadAll(Requirement.of(Catalog.class, "Downtown"));
lines.forEach(System.out::println);
```

For the catalog the explanation says, in order: the catalog was requested; the library capabilities handle it and the credentials factory declines; the library capabilities need `Credentials`; `Credentials` was requested, by the catalog; the credentials factory handles it; and how many providers each produced, in how many microseconds. That is the answer to "why was this wired to that?", and it costs nothing when nobody listens. `ResolutionListener.FactoryNodeCollector` collects the same events as a graph of factories and their dependencies.

8.5. Cycles

A factory may wait for a dependency. If the dependency waits for the factory, nothing can progress, and the resolution fails at once with the cycle named:

Listing 25. A cycle, reported

```
CapabilityFactory<Object, String> poultry = new CapabilityFactory<>() {

    @Override
    public boolean canHandle(Object requirement) {
        return "chicken".equals(requirement) || "egg".equals(requirement);
    }

    @Override
    public Flow.Publisher<CapabilityProvider<String>> create(
            Object requirement,
            Loader loader,
            Span span,
            Logger logger,
            Meter meter) {
        Object other = "chicken".equals(requirement) ? "egg" : "chicken";
        return Flows.fromFuture(loader
            .<String>loadAll(other) // Waits for the other, which waits for this one
            .thenApply(o -> CapabilityProvider.of(requirement + " from " + o)));
    }

};
assertThatThrownBy(() -> CapabilityLoader.of(poultry).loadAll("chicken"))
    .isInstanceOf(StallException.class)
```

```
    .hasMessageContaining("cycle: chicken -> egg -> chicken");
```

There is no timeout and no pass counter. Resolution runs on a pump (Chapter 9), which knows when every piece of work is waiting on another, and says what each one waits for.

8.6. Composition

Loaders compose:

- `CapabilityLoader.concat(loaders…)` lets every loader contribute, in order.
- `CapabilityLoader.fallback(loaders…)` stops at the first loader that provides anything.
- `CapabilityLoader.cached(loader)` remembers answers across calls, which a loader does not do by itself. Opt in where the same requirements recur and the answers are stable.

Composed loaders keep their resolutions separate. To let factories from several places satisfy each other's dependencies, compose their **sources** instead:

Listing 26. A local source in front of a global one

```
CapabilityFactorySource global = CapabilityFactorySource.of(CREDENTIALS);
CapabilityFactorySource local = CapabilityFactorySource.of(new LibraryCapabilities());
CapabilityLoader loader = CapabilityLoader.of(CapabilityFactorySource.concat(local, global));
assertThat(loader.<Catalog>loadOne(Requirement.of(Catalog.class, "Uptown"))).isNotNull();
```

The library capabilities in the local source find the credentials in the global one. `CapabilityFactorySource.serviceLoader()` is the global source in an application, and a local one in front of it overrides or adds capabilities for one task.

A loader is stateless and not closeable: the memory of a resolution lasts as long as the resolution, and disposing of factories is the business of their source. That is what makes the shared instance safe to share.

Chapter 9. Flows, the pump and the transformer

Matching finds the parts; wiring connects them. When the parts refer to each other, connecting them is a problem of order: an object cannot be given a reference to another that does not exist yet, and with cycles there is no order in which everything exists first. This chapter shows the three pieces the module solves it with: operators for `java.util.concurrent.Flow`, a pump that runs work deterministically and reports what is stuck, and a transformer that wires object graphs by subscription.

9.1. Flows

`Flow` is the JDK's interface for reactive streams: a publisher, a subscriber, and a subscription through which the subscriber requests items. The JDK has the interfaces and one publisher; `Flows` has sources, operators and terminals, named like `Collectors` and `Executors`:

Listing 27. Operators and terminals (`FlowTests.java`)

```
Flow.Publisher<Integer> numbers = Flows.fromIterable(List.of(1, 2, 3, 4, 5, 6, 7, 8));
Flow.Publisher<Integer> even = Flows.filter(numbers, n -> n % 2 == 0);
Flow.Publisher<String> labels = Flows.map(Flows.take(even, 3), n -> "#" + n);
assertThat(Flows.toList(labels).join()).containsExactly("#2", "#4", "#6"); ①

try (Stream<String> stream = Flows.toStream(labels)) { ②
    assertThat(stream.count()).isEqualTo(3);
}
```

① `toList` subscribes and completes a future with the items.

② `toStream` bridges to `java.util.stream`, for processing on the pull side.

Every operator is lazy, subscribing upstream once per downstream subscriber; respects demand, so `filter` requests one more item for every item it drops and `take` cancels upstream after the last; and turns an exception in a user function into `onError` downstream.

What is missing is missing on purpose: `flatMap`, `merge`, `zip`, schedulers and time operators. They are where reactive libraries get large and difficult, and on virtual threads they are ordinary blocking code. `concatMap`, sequential composition, is present because it is deterministic and capability resolution needs it.

9.2. The pump

A `Pump` is a serial work queue drained by one thread at a time. Its terminals block by pumping: a thread that waits for a result runs queued work instead of sleeping, as `ForkJoinTask.join()` and SWT's event loop do.

Listing 28. Pumping on the calling thread

```
Pump pump = new Pump();
List<String> log = new ArrayList<>();
CompletableFuture<String> result = new CompletableFuture<>();
pump.execute(() -> {
    log.add("a");
    pump.execute(() -> {
        log.add("c");
        result.complete("done");
    });
});
pump.execute(() -> log.add("b"));
assertThat(log).isEmpty(); ①
assertThat(pump.join(result)).isEqualTo("done"); ②
assertThat(log).containsExactly("a", "b", "c");
```

① Nothing runs until somebody pumps.

② `join` drains the queue on the calling thread until the future is complete. Tasks run in the order they were queued.

Without an executor, a pump is single-threaded and deterministic, which is what tests and resolutions need. With one, work queued while nobody pumps is drained there.

Work that cannot proceed yields: with a reason, `when(stage, description, action)`, which parks the work until the stage completes; or blindly, `submit(description, step)` with a step that returns false, which puts it back in the queue at a lower priority. Work that leaves the pump, an HTTP call or an answer from an AI model on another thread, is tracked with `track`, so that the pump knows it is still out.

9.3. Stalls

When everything in the queue is waiting and nothing is in flight, nothing can ever progress. The pump fails at once with a `StallException` that lists each waiting piece of work and what it waits for:

Listing 29. A stall, reported

```
Pump pump = new Pump();
pump.when(new CompletableFuture<>(), "waits for a reply nobody sends", (value, error) -> {});
assertThatThrownBy(pump::settle)
    .isInstanceOf(StallException.class)
    .hasMessageContaining("parked: waits for a reply nobody sends");
```

This is the check that wiring needs and seldom gets. A program that hangs says nothing; a pump that stalls says which parts waited for which.

9.4. Wiring by subscription

A `Transformer` maps sources to targets through a factory, and lets the factory ask for the targets of other sources while it creates its own. The example turns nodes that refer to each other by name into vertices that refer to each other by reference:

Listing 30. Sources and targets

```
record Node(String name, List<String> references) {}

static final class Vertex {

    final String name;
    final List<Vertex> connections = new ArrayList<>();

    Vertex(String name) {
        this.name = name;
    }

}
```

Listing 31. A cyclic graph, wired

```
Map<String, Node> nodes = Map.of(
    "a", new Node("a", List.of("b")),
    "b", new Node("b", List.of("a", "c")), // a and b refer to each other
    "c", new Node("c", List.of()));

Transformer<String, Vertex> transformer = new Transformer<>((name, context) -> {
    Vertex vertex = new Vertex(name);
    for (String reference: nodes.get(name).references()) {
        Flows.forEach(context.get(reference), vertex.connections::add); ①
    }
    return Flows.of(vertex); ②
});
```

```
Map<String, List<Vertex>> vertices = transformer.transform(List.of("a")); ③
Vertex a = vertices.get("a").get(0);
Vertex b = vertices.get("b").get(0);
assertThat(a.connections).containsExactly(b);
assertThat(b.connections.get(0)).isSameAs(a);
assertThat(vertices).containsKey("c");
```

① The vertex subscribes to the targets of each node it refers to, and connects them as they arrive.

② It emits itself at once, without waiting for its references.

③ Only `a` is requested; `b` and `c` are resolved because `a` and `b` asked for them.

There is no separate wiring phase. Each source is created once per transformation, and its targets are replayed to everyone who asks; because emitting does not wait for references, `a` and `b` wire to each other without deadlock.

A factory that does need another source's targets before it can emit its own can wait for them. If two sources wait for each other, the transformation fails with the cycle named, as a capability resolution does, because it runs on a pump too:

Listing 32. Waiting in a cycle

```
Transformer<String, String> transformer = new Transformer<>((name, context) -> {
    String other = name.equals("x") ? "y" : "x";
    return Flows.fromFuture(Flows.toList(context.get(other)) // Waits for the other
        .thenApply(targets -> name + targets));
});
assertThatThrownBy(() -> transformer.transform(List.of("x")))
    .isInstanceOf(StallException.class)
    .hasMessageContaining("cycle: x -> y -> x");
```

9.5. Factories from methods

A `ReflectiveFactory` builds a transformer factory from methods annotated with `@Mapping`. A source is matched to the methods whose parameter accepts it, most specific first:

Listing 33. Mapping methods

```
public static class Describer {

    @Mapping
    public String number(Number number) {
        return "number " + number;
```

```
    }

    @Mapping
    public String integer(Integer integer) { ①
        return "integer " + integer;
    }

    @Mapping
    public Flow.Publisher<String> text(CharSequence text, Context<Object, String> context) {
        return Flows.concat(Flows.of("text " + text), context.get(text.length())); ②
    }

}
```

① An `Integer` is a `Number` too; `integer` is more specific, so it wins.

② A method that takes the context can ask for the targets of other sources: the length of the text, described as a number.

Listing 34. Polymorphic dispatch

```
Transformer<Object, String> transformer = new Transformer<>(
    ReflectiveFactory.first(MethodHandles.lookup(), new Describer()));
Map<Object, List<String>> described = transformer.transform(List.of(1.5, 7, "abc"));
assertThat(described.get(1.5)).containsExactly("number 1.5");
assertThat(described.get(7)).containsExactly("integer 7");
assertThat(described.get("abc")).containsExactly("text abc", "integer 3");
```

`ReflectiveFactory.first` uses the most specific matching method, and `ReflectiveFactory.all` lets every matching method contribute. `@Mapping` also takes a priority, which ranks methods before specificity does.

The transformer is how the runtime converts one model into another, and the same factory can serve as a capability factory, a requirement being a source and its capabilities the targets (`CapabilityFactory.of`).

Chapter 10. Resource sets

An EMF resource set loads, saves and cross-references models, and it is a matching machine of its own. A URI is matched to a URI handler by its scheme, and to a resource factory by its extension or protocol; a namespace URI in a file is matched to a registered package. All of it has to be configured, and in a modular application the configuration belongs to the modules that bring the models, formats and schemes.

10.1. A configured resource set

`NasdanikaResourceSet.createAndConfigure()` creates a resource set and lets every module on the module path contribute to it:

- URI handlers provided as `URIHandler` services are added;
- adapter factories provided as `AdapterFactory` services are added;
- packages provided as `EPackage` services are registered by their namespace URIs;
- every `ResourceSetContributor` service is called.

The module itself contributes handlers for two URI schemes, and resource factories for XMI, the default for any extension, and for EMF's binary format, `.ebin`, and its compressed form, `.egz`:

Listing 35. Loading from the class path and from a module (`ResourceSetTests.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure(); ①

URI classpath = URI.createURI("classpath://book-data/library.ecore"); ②
EPackage library = (EPackage) resourceSet.getResource(classpath, true).getContents().get(0);
assertThat(library.getEClassifier("Book")).isNotNull();

URI module = URI.createURI("module://org.nasdanika.sdk.runtime.common/book-data/library.ecore");
Resource resource = resourceSet.getResource(module, true); ③
assertThat(((EPackage) resource.getContents().get(0)).getNsURI())
    .isEqualTo("urn:example:library");
```

① The resource set, configured from the module path.

② `classpath://` loads a resource with the thread's context class loader.

③ `module://` names a module in the authority and a resource in the path, and loads it from that module of the current module layer.

The module system encapsulates resources in a module's packages. A resource in a directory that is not a package, such as `book-data` here, can be read by anyone; a resource in a package, through `classpath://`, only if the package is opened to everyone.



> `ModuleURIHandlerFactory` reads `module://` resources with `Module.getResourceAsStream`, called from `common`, so a resource in a package of another module is found only if that package is opened to `common`; this should be stated in its Javadoc. It also finds the module through the handler's own module layer, `getClass().getModule().getLayer()`, which is null when `common` is on the class path, so `module://` URIs fail there with a `NullPointerException` rather than an `IOException`.

10.2. Contributors

A `ResourceSetContributor` configures a resource set: registers a resource factory, a package, a URI mapping, a URI handler.

Listing 36. A contributor

```
public static class XmlContributor implements ResourceSetContributor {

    @Override
    public void contribute(ResourceSet resourceSet, Span span, Logger logger) {
        resourceSet
            .getResourceFactoryRegistry()
            .getExtensionToFactoryMap()
            .put("xml", new XMLResourceFactoryImpl()); ①
    }

}
```

① Files with the extension `xml` become XML resources.

Declared as `provides ResourceSetContributor with ⋯` in a module's `module-info.java`, it is called for every resource set the runtime configures. It can also be called directly:

Listing 37. Contributing to a resource set

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
new XmlContributor().contribute(resourceSet); ①
Resource resource = resourceSet.createResource(URI.createURI("memory:/books.xml"));
assertThat(resource).isInstanceOf(org.eclipse.emf.ecore.xmi.XMLResource.class);
```

① `contribute(resourceSet)` runs the contribution in a span named after it, with the contributor's class and module as attributes.

10.3. Globals

A `NasdanikaResourceSet` can hold objects registered by URI, outside any resource. `getEObject` returns a registered object before it looks in resources:

Listing 38. A global object

```
NasdanikaResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
EClass book = EcoreFactory.eINSTANCE.createEClass();
book.setName("Book");
URI uri = URI.createURI("urn:example:library#Book");
resourceSet.registerGlobal(uri, book); ①
assertThat(resourceSet.getEObject(uri, false)).isSameAs(book);
```

① References to `urn:example:library#Book` resolve to this object, in any resource of the resource set.

Only registered objects are global: an object resolved through a resource stays in its resource, and goes when the resource is unloaded.

10.4. Resources from capabilities

`ContentsHandlingResourceFactory`, in the `capability.emf` package, creates resources whose contents are loaded and saved by handlers found as capabilities. The last segment of the URI is read as qualifiers, right to left: `family.ecore.groovy` has the qualifiers `groovy` and `ecore`, and handlers are requested for the most specific first. A resource format is then a capability that any module can provide, matched by the resource's name.



`ContentsHandlingResource` requires handlers with `Requirement.of(ResourceContentsHandler.class, handlerRequirement)`: the raw type, with the contents type, `EObject[].class`, carried in the payload. With the module's own matching it could require `ResourceContentsHandler<EObject[]>`, and handlers would be matched by their declared type arguments, as this book recommends everywhere else.

Chapter 11. Telemetry

A wiring that cannot be observed can be checked only by its results. Every match in this module runs in an OpenTelemetry span: configuring a resource set, contributing to it, loading and saving a resource, reading and writing its bytes, resolving a capability, creating a capability, transforming a source. This chapter shows where the spans go, and how to watch them.

11.1. One instance per unit of work

The module does not use `GlobalOpenTelemetry`. A unit of work, a command, an HTTP request or a test, builds an `OpenTelemetry` instance or is handed one, and makes it current, the way it makes a span current. The instance travels in the OpenTelemetry `Context`, next to the current span, so it crosses threads wherever the context does, and the pump propagates the context to its tasks (Chapter 9). Without one, `Telemetry.current()` returns the no-op instance, and instrumentation costs next to nothing.

A resource set can also be given an instance, `Telemetry.attach(resourceSet, otel)` or `NasdanikaResourceSet.createAndConfigure(otel)`, for work that runs outside any context, such as resolving a proxy long after the resource set was configured. The context, when it has an instance, wins.

11.2. Watching spans live

An OpenTelemetry SDK exports a span when it ends, which is too late to report the progress of a long operation. `PublishingOpenTelemetry` wraps another instance, an SDK or the no-op one, and publishes what happens to its spans as it happens, as `SpanEvent` records: started, attribute set, status set, exception recorded, ended, and the log records emitted in a span.

Listing 39. A subscriber to span events (`TelemetryTests.java`)

```
static final class Watcher implements Flow.Subscriber<SpanEvent> {

    final List<String> lines = Collections.synchronizedList(new ArrayList<>());

    @Override
    public void onSubscribe(Flow.Subscription subscription) {
        subscription.request(Long.MAX_VALUE);
    }
```

```
    @Override
    public void onNext(SpanEvent event) {
        switch (event) { ①
            case SpanEvent.Started started -> lines.add("start " + started.name());
            case SpanEvent.Ended ended -> lines.add("end " + ended.span().getName());
            case SpanEvent.LogEmitted log -> lines.add("log " + log.record().bodyAsString());
            default -> {}
        }
    }

    @Override
    public void onError(Throwable throwable) {
        throw new AssertionError(throwable);
    }

    @Override
    public void onComplete() {
        // Nothing to release
    }

}
```

① `SpanEvent` is a sealed interface of records, so a subscriber matches the events it cares about.

Listing 40. A unit of work, watched

```
try (PublishingOpenTelemetry otel = publishing(); Scope scope = Telemetry.makeCurrent(otel)) {
    Watcher watcher = new Watcher();
    otel.subscribe(watcher); ①

    OpenTelemetry current = Telemetry.current(); ②
    Telemetry.inSpan(Telemetry.tracer(current), "import", null, span -> { ③
        Telemetry.log(Telemetry.logger(current), Severity.INFO, null, "two files", null);
        return null;
    });

    assertThat(watcher.lines).containsExactly("start import", "log two files", "end import");
}
```

① Subscribing to the instance, to watch every span of the unit of work. `within(span)` publishes the events of one span and its descendants.

② The instance current for this unit of work.

③ `inSpan` starts a span, makes it current, runs the body, records a failure, and ends the span.

Instrumented code must never block or fail because somebody is watching it, so events are offered to subscribers, not forced on them: a subscriber whose buffer is full loses events, and the loss is counted. With no subscribers, events are not even created. The tests in this book deliver events on the calling thread, so that they can assert on them at once; by default they are

delivered on virtual threads.

Log records go to the OpenTelemetry Logs API directly, correlated with the current span; the runtime uses no logging framework, and where a record goes is decided by whoever runs the unit of work. Span events, `Span.addEvent`, are not used, because OpenTelemetry is moving events onto the Logs API.

11.3. Resources

A resource set configured with an instance reports what it does:

Listing 41. Saving a resource, watched

```
try (PublishingOpenTelemetry otel = publishing()) {
    Watcher watcher = new Watcher();
    otel.subscribe(watcher);

    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure(otel); ①
    URI uri = URI.createFileURI(dir.resolve("library.xmi").toString());
    Resource resource = resourceSet.createResource(uri);
    EPackage library = EcoreFactory.eINSTANCE.createEPackage();
    library.setName("library");
    resource.getContents().add(library);
    resource.save(null);

    assertThat(watcher.lines).contains(
        "start ResourceSetContributor.configure",
        "start Resource.save",
        "start URIHandler.write", ②
        "end URIHandler.write",
        "end Resource.save");
}
```

① The instance is attached to the resource set.

② The URI handlers are wrapped, so every stream opened through them is a span from opening to closing, with the URI, the handler and the byte count as attributes, and progress reported while bytes are read or written.

Resources created by the module's factories report `Resource.load`, `Resource.save` and `Resource.unload` spans, with the byte count and the number of root objects.

11.4. Propagation

A unit of work that continues another one, a pipeline step started by another, an HTTP request from a page, takes its parent from a carrier: a map of `traceparent` and `baggage` values in the W3C format. `Telemetry.extract` reads one from any map, with keys in upper case, as in environment variables, or in lower case, as in request headers and parameters:

Listing 42. A parent from the environment

```
OpenTelemetry otel = OpenTelemetry.propagating(Telemetry.w3cPropagators());
Map<String, String> environment = Map.of(
    "TRACEPARENT", "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01");
Context parent = Telemetry.extract(otel, Context.root(), environment); ①
assertThat(Span.fromContext(parent).getSpanContext().getTraceId())
    .isEqualTo("4bf92f3577b34da6a3ce929d0e0e4736");
```

① The context now has a remote parent span; spans started in it join the caller's trace.

`Telemetry.extractFromEnvironment` does the same with the process's environment variables, so a command started by a pipeline joins the pipeline's trace.

11.5. Keys and names

`Telemetry` defines the attribute keys and event names the runtime uses, all in the `nasdanika.` namespace: the URI and type of a resource, the number of bytes read or written, the requirement and factory of a capability, the source of a transformation, the kind of a contribution to a resource set, and the progress of an operation, among others. Use them in your own instrumentation where they fit, so that tools reading the telemetry need to know one vocabulary.

Part III: Looking Ahead

What is built on this module, and what is planned for it.

Chapter 12. What is built on it, and what comes next

12.1. Built on it

Every module of the runtime depends on `common`, and most of them match through it.

Command wiring. The runtime's command line, in the `cli` module, assembles a tree of commands from modules that know nothing of each other. A command declares what it needs from its parent with `@Parent` members and what it offers its sub-commands with `@SubCommand` members, by declared generic types; the tree is matched first by declarations, with `Adapters.isAdaptable`, before any command exists, and then by instances, with `Adapters.adapt`. The command that saves models declares a `@Parent` field of type `ExecutionSupplier<? extends Collection<? extends EObject>>`, so any command that can be adapted to a supplier of a collection of model objects, whatever loads them, can be its parent.

Models. The runtime's models register their packages as services, and are loaded and saved through resource sets configured by this module, with formats such as JSON and YAML contributed by other modules.

Capabilities. The command line and the Groovy DSL provide capability factories, and modules built on the runtime provide their own, all matched by type and loaded by the same loader.

12.2. Planned

PLANNED:

- **Specificity by assignability.** Ordering candidates by the declared types they provide, not only by their erased classes, so that `Supplier<Book>` comes before `Supplier<Publication>` because it is assignable to it (Chapter 7).
- **Generic contents handlers.** Resource contents handlers required by their generic type (Chapter 10).
- **A diagram of a resolution.** `ResolutionListener.FactoryNodeCollector` already collects the graph of factories and their dependencies; rendered with the runtime's diagram model and layout,

it would show the wiring of an application as a picture.

- **Wiring as a model.** The explanation of a resolution and the spans of a transformation, saved as models like the rest of the runtime’s telemetry, so that the wiring of one run can be compared with the wiring of the next.

Appendix A: Packages and types

The main types of each package. Every package is exported.

A.1. `org.nasdanika.sdk.runtime.common`

Type	Purpose
`Types`	Generic type operations: `isAssignable`, `getTypeArgument`, `getElementType`, `isGround`, `erase`, `getFieldType`, `getReturnType`, `getParameterTypes`, `box`, `specificity`
`TypeToken<T>`	Captures a generic type as an anonymous subclass
`Adaptable`	An object that can be asked to adapt to a type, with context; a default implementation over `@Adapter` members
`@Adapter`	Marks a field or method whose declared type an object adapts to
`Adapters`	`adapt` for any object, and `isAdaptable` for a class, from declarations only
`NasdanikaResourceSet`	A resource set configured from the module path, with global objects
`Component`, `Closeable`	Lifecycle interfaces: a `close` that throws no checked exception
`@Description`	Documentation of a command, option or parameter
`NasdanikaException`	The runtime's unchecked exception
`Util`	Grouping with null keys, and the lineage of a class

A.2. `org.nasdanika.sdk.runtime.common.capability`

Type	Purpose
`Requirement`	A ground type with payloads; a record
`CapabilityFactory<R, C>`	Handles requirements and creates providers; `Loader` resolves its dependencies
`CapabilityProvider<T>`	Publishes capabilities
`CapabilityLoader`	Resolves requirements; `of`, `getInstance`, `concat`, `fallback`, `cached`, `withListener`
`CapabilityFactorySource`	Where factories come from: `serviceLoader`, `of`, `concat`, `filtered`

Type	Purpose
ReflectiveCapabilityFactory	A factory assembled from @Capability methods
ResolutionListener, ResolutionEvent	Events of a resolution; explaining and FactoryNodeCollector
TransformerCapabilityFactory	A transformer factory as a capability factory

A.3. org.nasdanika.sdk.runtime.common.capability.emf

Type	Purpose
ContentsHandlingResourceFactory, ContentsHandlingResource	Resources loaded and saved by handlers found as capabilities
ResourceContentsHandler<T>	Loads and saves contents; Requirement and Order select handlers

A.4. org.nasdanika.sdk.runtime.common.flow

Type	Purpose
Flows	Sources, operators and terminals for java.util.concurrent.Flow
Pump	A serial work queue whose terminals block by pumping, with stall detection
PumpedPublisher	A publisher drained on its own pump, with blocking terminals
StallException, PumpException	What stopped a pump
Transformer<S, T>	Maps sources to targets, wiring by subscription
Transformation	One transformation on one pump
ReflectiveFactory	A transformer factory assembled from @Mapping methods
ConfinedSubscriber	A subscriber whose signals run on a pump

A.5. org.nasdanika.sdk.runtime.common.services

Type	Purpose
`ResourceSetContributor`	Configures resource sets; `configure` runs every contribution on the module path
`ClassPathURIHandlerFactory`, `ModuleURIHandlerFactory`	Handlers for `classpath://` and `module://`
`XMIResourceFactoryContributor`	XMI for any extension, instrumented
`BinaryResourceFactoryContributor`, `GzipBinaryResourceFactoryContributor`	EMF binary resources, `.ebin` and `.egz`
`EObjectSupplier`, `EModelElementSupplier`	Service interfaces that supply model objects

A.6. `org.nasdanika.sdk.runtime.common.telemetry`

Type	Purpose
`Telemetry`	The current instance, attachment to resource sets, `inSpan`, `log`, propagation, and the runtime's attribute keys and event names
`PublishingOpenTelemetry`	An `OpenTelemetry` whose spans and log records can be watched live
`PublishingTracerProvider`, `PublishingTracer`, `PublishingSpan`, `PublishingLoggerProvider`, `PublishingLogger`	Its parts
`SpanEvent`, `LogRecord`	What is published
`TelemetryURIHandler`	Wraps a URI handler so that streams are spans
`ResourceTelemetry`	Spans and records for resource load, save and unload

Appendix B: Building this book

This book is AsciiDoc in `common/src/docs/asciidoc`, built with the module after its tests have passed:

```
mvn -pl common verify                                # the module, its tests and this book
mvn -pl common verify -Dzephyr.editor-notes=false    # without editor's notes
mvn -pl common verify -Dasciidoctor.skip=true        # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the module's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/common/book`, and of the module's `module-info.java`. The tests use resources in `src/test/resources/book-data`, a directory that is not a package, so that the module system does not encapsulate them. A listing line holds at most 96 characters, and a warning fails the build.

Appendix C: Glossary

Adaptation

Asking an object for something of a type, with optional context, and getting it or null (Chapter 7).

Capability

An object provided for a requirement by a capability factory (Chapter 8).

Capability factory

Handles some requirements and creates providers of capabilities for them; may require other capabilities.

Capability loader

Resolves a requirement against a source of factories, with their dependencies.

Declaration

What a program says about a class, field or method, generic types included, as the compiler checked it and the class file keeps it. What the module matches on.

Erasure

The removal of type arguments from types at compile time. Instances carry no type arguments; declarations keep them.

Ground type

A type with no type variables in it, which can therefore be required or provided.

Matching

Deciding whether something that is offered fits something that is needed, by a rule.

Payload

A value of a requirement other than its type, matched to the parameters of a capability method in order.

Pump

A serial work queue whose waiting threads run queued work, and which reports a stall when nothing can progress (Chapter 9).

Requirement

What is needed: a ground type and payloads, or any object a factory understands.

Resource set

EMF's container of resources, which matches URIs to handlers and factories, and namespaces to packages (Chapter 10).

Sensory bias

In evolutionary biology, a preference that evolved in one context and is exploited in another; here, Java developers' fluency with generics, used for wiring (Chapter 2).

Signature attribute

The part of a class file that keeps generic declarations.

Stall

The state in which every piece of work on a pump waits for another and nothing is in flight; reported by a `StallException` with the waits named.

Type token

An anonymous subclass of `TypeToken` that captures a generic type for reflection.

Unit of work

A command, a request or a test, with its own OpenTelemetry instance current (Chapter 11).

Wiring

The references between parts of a program that matching produced.